

COS 350 Lecture 7

unix text file searching: grep

case study: cp

creat()

write()

errno, perror()

cost of system calls

large buffers

NAME

grep, egrep, fgrep, rgrep - print lines matching a pattern

SYNOPSIS

grep [OPTIONS] PATTERN [FILE...]

DESCRIPTION

grep searches the named input FILEs (or standard input if no files are named) for lines containing a match to the given PATTERN. By default, grep prints the matching lines.

In addition, three variant programs egrep, fgrep and rgrep are available. egrep is the same as grep -E. fgrep is the same as grep -F. rgrep is the same as grep -r.

OPTIONS

- i Ignore case distinctions, so that characters that differ only in case match each other.
- v Invert the sense of matching, to select non-matching lines.

Examples:

1. `grep class /usr/share/dict/words`

```
class
class's
classed
...
```

2. `grep '^q' words`

```
qua
quack
quackery
...
```

words

```
a
aback
abacus
abalone
abandon
abase
abash
```

3. `egrep -i '^[a-f]{4}$' /usr/share/dict/words`

```
abbe  
abed  
babe  
bade  
bead  
beef  
cafe  
cede  
Dada  
Dade  
dead  
deaf  
deed  
face  
fade  
feed
```

Operator Type	Examples	Description
Literal Characters	a A y 6 % @	Letters, digits and many special characters match exactly
	\\$ \^ \+ \\ \?	Precede other special characters with \
Anchors	^	Start of line
	\$	End of line
Character groups	[aeiouAEIOU]	any 1 character listed (vowel)
	[^aeiouAEIOU]	any character not listed (non vowel)
	[a-fA-F0-9]	any hex character (0 to 9 or a to f)
	.	any character at all
Counts apply to previous element	+	1 or more ("some") (egrep only)
	*	0 or more ("perhaps some")
	?	0 or 1 ("perhaps a")
	{4}	repeat number of time (egrep only)
Alternation		either, or (egrep only)
Grouping	()	group for count (egrep only)

Operator Type	Examples	Description
Literal Characters	a A y 6 % @	Letters, digits and many special characters match exactly
	\\$ \^ \+ \\ \?	Precede other special characters with \
Anchors	^	Start of line
	\$	End of line
Character groups	[aeiouAEIOU]	any 1 character listed (vowel)
	[^aeiouAEIOU]	any character not listed (non vowel)
	[a-fA-F0-9]	any hex character (0 to 9 or a to f)
	.	any character at all
Counts apply to previous element	+	1 or more ("some") (egrep only)
	*	0 or more ("perhaps some")
	?	0 or 1 ("perhaps a")
	{4}	repeat number of time (egrep only)
Alternation		either, or (egrep only)
Grouping	()	group for count (egrep only)

1. Find words ending in “tch”, such as “watch”

Operator Type	Examples	Description
Literal Characters	a A y 6 % @	Letters, digits and many special characters match exactly
	\\$ \^ \+ \\ \?	Precede other special characters with \
Anchors	^	Start of line
	\$	End of line
Character groups	[aeiouAEIOU]	any 1 character listed (vowel)
	[^aeiouAEIOU]	any character not listed (non vowel)
	[a-fA-F0-9]	any hex character (0 to 9 or a to f)
	.	any character at all
Counts apply to previous element	+	1 or more ("some") (egrep only)
	*	0 or more ("perhaps some")
	?	0 or 1 ("perhaps a")
	{4}	repeat number of time (egrep only)
Alternation		either, or (egrep only)
Grouping	()	group for count (egrep only)

2. Find words with 4 vowels in a row, such as “aqueous”

Operator Type	Examples	Description
Literal Characters	a A y 6 % @	Letters, digits and many special characters match exactly
	\\$ \^ \+ \\ \?	Precede other special characters with \
Anchors	^	Start of line
	\$	End of line
Character groups	[aeiouAEIOU]	any 1 character listed (vowel)
	[^aeiouAEIOU]	any character not listed (non vowel)
	[a-fA-F0-9]	any hex character (0 to 9 or a to f)
	.	any character at all
Counts apply to previous element	+	1 or more ("some") (egrep only)
	*	0 or more ("perhaps some")
	?	0 or 1 ("perhaps a")
	{4}	repeat number of time (egrep only)
Alternation		either, or (egrep only)
Grouping	()	group for count (egrep only)

3. Find words at least 15 characters long

Operator Type	Examples	Description
Literal Characters	a A y 6 % @	Letters, digits and many special characters match exactly
	\\$ \^ \+ \\ \?	Precede other special characters with \
Anchors	^	Start of line
	\$	End of line
Character groups	[aeiouAEIOU]	any 1 character listed (vowel)
	[^aeiouAEIOU]	any character not listed (non vowel)
	[a-fA-F0-9]	any hex character (0 to 9 or a to f)
	.	any character at all
Counts apply to previous element	+	1 or more ("some") (egrep only)
	*	0 or more ("perhaps some")
	?	0 or 1 ("perhaps a")
	{4}	repeat number of time (egrep only)
Alternation		either, or (egrep only)
Grouping	()	group for count (egrep only)

4. Find words with all 5 vowels in order, such as “facetious”

Case study: `cp` copy a file

usage: `cp srcfile destfile`

NAME

open, creat - open and possibly create a file or device

SYNOPSIS

```
#include <sys/types.h>
```

```
#include <sys/stat.h>
```

```
#include <fcntl.h>
```

```
int open(const char *pathname, int flags);
```

```
int open(const char *pathname, int flags, mode_t mode);
```

```
int creat(const char *pathname, mode_t mode);
```

...



file access permissions
use value 0644 for
now

RETURN VALUE

open() and creat() return the new file descriptor, or -1 if an error occurred (in which case, **errno** is set appropriately).

NAME

write - write to a file descriptor

SYNOPSIS

```
#include <unistd.h>
```

```
size_t write(int fd, const void *buf, size_t count);
```

DESCRIPTION

write() writes up to count bytes from the buffer pointed buf to the file referred to by the file descriptor fd.

RETURN VALUE

On success, the number of bytes written is returned (zero indicates

nothing was written). On error, -1 is returned, and **errno** is set appropriately.

```
/* cp1.c  version 1 of cp
 * uses read and write with tunable buffer size
 *
 * usage: cp1 src dest
 */
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
```

```
#define BUFFERSIZE      4096
#define COPYMODE        0644
```

```
void oops(char *s1, char *s2)
{
    fprintf(stderr, "Error: %s ", s1);
    perror(s2);
    exit(1);
}
```

```
main(int ac, char *av[]) {
    int      in_fd, out_fd, n_chars;
    char      buf[BUFFERSIZE];

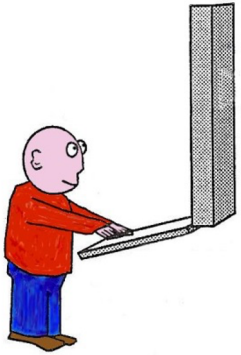
    if ( ac != 3 ){      /* check args          */
        fprintf( stderr, "usage: %s source destination\n", *av);
        exit(1);
    }

    /* open files */
    if ( (in_fd=open(av[1], O_RDONLY)) == -1 )
        oops("Cannot open ", av[1]);
    if ( (out_fd=creat( av[2], COPYMODE)) == -1 )
        oops("Cannot creat", av[2]);

    /* copy the file */
    while ( (n_chars = read(in_fd , buf, BUFFERSIZE)) > 0 )
        if ( write( out_fd, buf, n_chars ) != n_chars )
            oops("Write error to ", av[2]);

    if ( n_chars == -1 )
        oops("Read error from ", av[1]);

    /* close files */
    if ( close(in_fd) == -1 || close(out_fd) == -1 )
        oops("Error closing files","");
}
```



demo cp1

words

errors:

No such file

Permission denied

NAME

perror, errno - print system error

SYNOPSIS

```
#include <stdio.h>
```

```
void perror(const char *s);  
#include <errno.h>  
int errno;
```

DESCRIPTION

The `perror()` function produces a message on the standard error output (file descriptor 2) describing the last error encountered during a call to a system or library function. The argument string `s` is printed, followed by a colon and a blank, followed by the message and a NEWLINE character. If `s` is a null pointer or points to a null string, the colon is not printed. The argument string should include the name of the program that incurred the error. The error number is taken from the **external variable `errno`**, which is set when errors occur but not cleared when non-erroneous calls are made.

```
oops("Cannot open ", av[1]);  
  
void oops(char *s1, char *s2)  
{  
    fprintf(stderr, "Error: %s ", s1);  
    perror(s2);  
    exit(1);  
}  
  
Error: Cannot open f2: Permission denied
```


excerpt from errno.h

```
// Error codes
#define EPERM    1           // Not super-user
#define ENOENT   2           // No such file or directory
#define ESRCH    3           // No such process
#define EINTR    4           // interrupted system call
#define EIO      5           // I/O error
#define ENXIO    6           // No such device or address
#define E2BIG    7           // Arg list too long
#define ENOEXEC  8           // Exec format error
#define EBADF    9           // Bad file number
#define ECHILD   10          // No children
#define EAGAIN   11          // Resource temporarily
unavailable
#define ENOMEM   12          // Not enough core
#define EACCES   13          // Permission denied
#define EFAULT   14          // Bad address
#define ENOTBLK  15          // Block device required
#define EBUSY    16          // Mount device busy
#define EEXIST   17          // File exists
#define EXDEV    18          // Cross-device link
#define ENODEV   19          // No such device
```

The following table shows execution time for `cp1` copying a 5-megabyte file with various values of `BUFFERSIZE`:

buffer size	execution time in seconds
1	50.29
4	12.81
16	3.28
64	0.96
128	0.56
256	0.37
512	0.27
1024	0.22
2048	0.19
4096	0.18
8192	0.18
16384	0.18

System calls consume time. A program that makes a lot of system calls runs slower and takes time other users might want to use.